

```
/*
```

```
=====
```

ES6 LMS Coding Assignment – Set 2

Course: Web Technology / JavaScript ES6

```
=====
```

```
*/
```

```
// ----- Dummy Data -----
```

```
const prices = [1200, 450, 799, 1500, 299, 999];
```

```
const product = {  
  name: "Laptop",  
  id: 501,  
  brand: "Lenovo",  
  category: "Electronics",  
  details:{  
    warranty:"2 Years",  
    origin:"India"  
  }  
};
```

```
const customer = {  
  username: "rahul92",  
  premium: true  
};
```

```
// =====
```

```
// Q1
```

```
// Using arrow functions and array methods,  
// return prices greater than 500.  
// =====  
function expensiveItems(arr){  
  
    return arr.filter(price => price > 500);  
  
}
```

```
// =====  
// Q2  
// Using array destructuring,  
// return the sum of the first and last price.  
// =====  
function edgePriceSum(arr){  
  
    const [first, , , , last] = arr;  
  
    return first + last;  
  
}
```

```
// =====  
// Q3  
// Using object destructuring,  
// return string:  
// "Laptop from India belongs to Electronics category"  
// =====  
function productSummary(obj){
```

```
const {
  name,
  category,
  details: { origin }
} = obj;

return `${name} from ${origin} belongs to ${category} category`;

}
```

```
// =====
// Q4
// Using template literals,
// return product code:
// "PRD-501-Laptop"
// =====
function generateProductCode(obj){

  return `PRD-${obj.id}-${obj.name}`;

}
```

```
// =====
// Q5
// Using rest operator,
// return the highest price among all parameters
// =====
function highestPrice(...nums){
```

```

    return Math.max(...nums);

}

// =====
// Q6
// Using spread operator,
// merge product and customer and
// add property purchaseStatus : "CONFIRMED"
// =====
function createOrder(obj, cust){

    return {
        ...obj,
        ...cust,
        purchaseStatus: "CONFIRMED"
    };

}

// =====
// Q7
// Return a Promise resolving after 2 second:
// "Customer rahul92 placed an order"
// =====
function orderMessage(cust){

    return new Promise((resolve) => {

```

```

    setTimeout(() => {
      resolve(`Customer ${cust.username} placed an order`);
    }, 2000);

  });

}

// =====
// Q8
// Increase every price by tax percentage.
// Default tax = 10%.
// =====
function applyTax(arr, tax = 10){

  return arr.map(price => price + (price * tax / 100));

}

// =====
// Q9
// Create a Promise that resolves AFTER 2 seconds.
//
// If total price > 3000
// resolve -> "High Value Order"
//
// otherwise reject -> "Low Value Order"
// =====

```

```

function evaluateOrder(arr){

    return new Promise((resolve, reject) => {

        setTimeout(() => {

            const total = arr.reduce((sum, price) => sum + price, 0);

            if(total > 3000){
                resolve("High Value Order");
            }
            else{
                reject("Low Value Order");
            }

        }, 2000);

    });

}

// =====
// Q10
// Consume evaluateOrder() using then() and catch()
// and print result in UPPERCASE.
// =====

function orderReport(arr){

    evaluateOrder(arr)

        .then(res => console.log("Q10:", res.toUpperCase()))

```

```

        .catch(err => console.log("Q10:", err.toUpperCase()));

    }

// =====
// DRIVER FUNCTION (DO NOT MODIFY)
// =====
function driver(){

    console.log("Q1:", expensivItems(prices));

    console.log("Q2:", edgePriceSum(prices));

    console.log("Q3:", productSummary(product));

    console.log("Q4:", generateProductCode(product));

    console.log("Q5:", highestPrice(1200,450,799,1500,299));

    console.log("Q6:", createOrder(product,customer));

    orderMessage(customer).then(res=>console.log("Q7:",res));

    console.log("Q8:", applyTax(prices));

    evaluateOrder(prices)
        .then(res=>console.log("Q9:",res))
        .catch(err=>console.log("Q9:",err));

```

```
orderReport(prices);
```

```
}
```

```
driver();
```